

graphlex.facts() — feature specification for finalization

graphlex project (graph $\rightarrow$ deterministic, LLM-interpretable language)

2026-06-15 (draft for collaborator review)

Purpose. `facts(G)` is the single deterministic contract of graphlex: it maps a NetworkX graph to exact structural quantities (*no LLM*). This document lists every candidate feature so we can finalize the set. Please mark the **Keep?** box (or strike a feature / add one).

The non-negotiable design rule. The same finalized feature set must be carried, identically, by all four of:

1. **facts()** — computed values;
2. **verbalize()** — the text the *LLM* sees;
3. the **logreg feature vector** — what the classical (no-LLM) baseline is trained on;
4. the **canonical reference** (`src/graphfm/eval/properties.py`, the original NetworkStats vector).

The headline comparison (“LLM over verbalized features vs. logreg over the same features”) is only coherent if (1)–(3) are byte-for-byte the same feature set. They are *not* today; this spec is the fix. Implementation should declare each feature *once* in a shared registry from which `facts()`, `verbalize()`, and the logreg vector all derive, plus a `verbalization_version` stamp recorded in every result file.

Status legend. CUR = already in `facts()`; PORT = already computed in `properties.py`, needs porting; NEW = net-new.

feature	definition	status	keep?
A. Size & connectivity			
<code>n_nodes</code>	number of nodes n	CUR	☐
<code>n_edges</code>	number of edges m	CUR	☐
<code>density</code>	$2m/[n(n-1)]$	CUR	☐
<code>n_components</code>	number of connected components	CUR	☐
B. Degree distribution (incl. scale-free indicators)			
<code>mean_degree</code>	$\langle k \rangle$	CUR	☐
<code>max_degree</code>	$\max_i k_i$	CUR	☐
<code>degree_std</code>	standard deviation of degree	CUR	☐
<code>max_over_mean_degree</code>	$\max k / \langle k \rangle$ (hub dominance)	CUR	☐
<code>degree_skewness</code>	3rd standardized moment of the degree distribution	PORT	☐
<code>degree_kurtosis</code>	4th standardized moment (tail heaviness)	PORT	☐
<code>degree_gini</code>	Gini coefficient of degrees (inequality / heavy tail)	PORT	☐
<code>powerlaw_alpha</code>	power-law exponent (Clauset–Shalizi–Newman fit)	NEW	☐
<code>powerlaw_xmin</code>	lower cutoff of the fitted power law	NEW	☐
<code>powerlaw_ks</code>	KS distance of the fit; <i>flag “unreliable, n small” for $n \lesssim 50$</i>	NEW	☐

feature	definition	status	keep?
C. Clustering & cohesion			
avg_clustering	mean local clustering coefficient	CUR	☐
transitivity	$3 \times (\text{triangles}) / (\text{connected triples})$	CUR	☐
n_triangles	number of triangles	PORT	☐
n_squares	number of 4-cycles	PORT	☐
D. Distances (largest connected component)			
avg_path_length	mean shortest-path length	CUR	☐
diameter	maximum shortest-path length	CUR	☐
radius	minimum node eccentricity	NEW	☐
E. Mixing & community			
degree_assortativity	Pearson correlation of endpoint degrees	CUR	☐
n_communities	number of greedy-modularity communities	CUR	☐
modularity	Newman modularity Q of that partition (already a config-null contrast)	PORT	☐
F. Cores & spectral			
max_kcore	degeneracy (largest k with non-empty k -core)	PORT	☐
spectral_gap	algebraic connectivity (2nd-smallest Laplacian eigenvalue)	PORT	☐
G. Cycles / rings			
n_cycles	cyclomatic number $m - n + c$	CUR	☐
ring_sizes	histogram of minimum-cycle-basis ring lengths	CUR	☐
H. Centrality — graph-level scalar summaries			
mean_betweenness	mean node betweenness centrality	PORT	☐
max_betweenness	maximum node betweenness (top broker)	NEW	☐
mean_closeness	mean closeness centrality	PORT	☐
mean_eigenvector	mean eigenvector centrality	PORT	☐
max_eigenvector	maximum eigenvector centrality	NEW	☐
I. Node-level (per node; verbalized as named top-k, summarized as scalars for logreg)			
degree	node degree	CUR	☐
degree_centrality	degree $/(n - 1)$	PORT	☐
clustering	per-node local clustering	PORT	☐
betweenness	per-node betweenness	CUR	☐
closeness	per-node closeness	PORT	☐
eigenvector	per-node eigenvector centrality	CUR	☐
pagerank	per-node PageRank	PORT	☐
kcore_number	per-node k -core index	PORT	☐
mean_neighbor_degree	average degree of a node's neighbors	PORT	☐
J. Null-model contrasts (named baselines; analytic where possible, else seeded rewiring)			
er_clustering_ratio	observed clustering / ER baseline (= density)	NEW	☐
config_clustering_ratio	observed clustering / configuration-model expectation	NEW	☐
config_assortativity_delta	$(\langle k^2 \rangle - \langle k \rangle)^2 / (n \langle k \rangle^3)$; report z		
	observed assortativity minus configuration-model expectation (≈ 0)	NEW	☐
smallworld_sigma	$\sigma = (C/C_{\text{rand}})/(L/L_{\text{rand}})$	NEW	☐
smallworld_omega	$\omega = L_{\text{rand}}/L - C/C_{\text{latt}}$	NEW	☐

Null models — what we mean. A null model is not unique: it is a randomized ensemble that fixes some properties and shuffles the rest; the choice of what to fix defines the question. graphlex never claims “the” null — it reports each beyond-chance statistic against *named* standard nulls and

states which: **ER** $G(n, m)$ (fix node+edge counts) and the **configuration model** (fix the exact degree sequence — the principled default, since it controls for the degree distribution). Compute analytically where a closed form exists (ER clustering = density; Newman’s configuration-model clustering; modularity Q is already a configuration-null contrast), else by seeded degree-preserving rewiring (~ 200 double-edge swaps) giving mean $\pm$ std and a z -score. At small n the null variance is large — always emit the caveat.

Small- n honesty. Several features (power-law fit, null-model z -scores, modularity) are unreliable on tiny graphs. The finalized verbalizer must attach an explicit caveat (e.g. “ $n = 30$ — too small to confirm a power law”) rather than report a bare number.

To finalize: tick $\square$, strike rejects, add anything missing, and flag any feature that should be *node-level only* vs. *graph-level scalar* vs. both.